TR Fastenings Multimedia CMS

Local deployment on your WiFi network (Docker)

Runs the web CMS and its PostgreSQL database in Docker on one Windows PC. Any phone, laptop or screen player on the same WiFi can then open the CMS in a browser, and the desktop screen players (XOS.exe) can show content from it.

Part	What it does	Where it runs
app (Next.js)	Web CMS, upload and screen assignment, API. Port 3000.	Docker container on the host PC
db (PostgreSQL 16)	Stores sessions, documents, screens and users. Not exposed to WiFi.	Docker container on the host PC
Media files	Uploaded images, videos and PDFs are stored in Cloudinary (cloud).	Internet (needs internet access)
Realtime (optional)	Pushes change notices to screen players through Ably.	Internet (only if ABLY keys are set)

> **Tip:** The host PC must stay on, awake and connected to WiFi for others to use the CMS. Uploads and the screen players need internet access, because media is stored in Cloudinary.

Step 1. Check what you need

Item	Notes
Windows 10/11 PC (the host)	Stays on during working hours. Wired ethernet is more reliable than WiFi for a server.
Docker Desktop for Windows	Install from docker.com. Must be running (whale icon in the taskbar).
Project folder	The repository, including **Dockerfile** and **docker-compose.yml**.
Cloudinary account	Cloud name, API key and API secret, from the Cloudinary dashboard.
Ably keys (optional)	Needed only for instant screen updates. Without them, players pick up changes on restart.
WiFi access	Host and client devices on the same network, with no client isolation (see Troubleshooting).

Step 2. Set a fixed IP address for the host PC

Other devices find the CMS through the host PC's IP address. If that address changes, the screen players and bookmarks stop working. Give the PC a fixed address:

a. Open the router admin page (usually `192.168.1.1` or `192.168.0.1`, printed on the router label).

b. Find the host PC in the DHCP client list and reserve its IP address, so it always gets the same one. Alternatively, set a static IPv4 address on the PC's WiFi adapter.

c. Find the current address on the host PC:

```
ipconfig
```

Look for **IPv4 Address** under **Wi-Fi**. The example in this guide uses `192.168.29.239`. Replace it with your own address everywhere it appears.

Step 3. Set the WiFi network to Private

Windows blocks incoming connections on **Public** networks. Open **Settings** → **Network & internet** → **Wi-Fi** → **(your network)** and set **Network profile type** to **Private network**.

Step 4. Allow port 3000 through Windows Firewall

Open **PowerShell as Administrator** and run this once:

```
New-NetFirewallRule -DisplayName "TR CMS (port 3000)" `
  -Direction Inbound -Protocol TCP -LocalPort 3000 `
  -Action Allow -Profile Private
```

Only **Private** networks are opened. Remove the rule later with `Remove-NetFirewallRule -DisplayName "TR CMS (port 3000)"`.

Step 5. Make Docker start automatically

Open **Docker Desktop** → **Settings** → **General** and turn on **Start Docker Desktop when you log in**. The containers use `restart: always`, so they come back after a reboot once Docker is running.

Step 6. Create the .env file

In the project root, create a file named `.env` (no other extension). Fill in your own values:

```
CLOUDINARY_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_SECRET_KEY=your_api_secret

# Optional: instant screen updates
ABLY_API_KEY=your_ably_publish_key
ABLY_SUBSCRIBE_KEY=your_ably_subscribe_key

# DATABASE_URL and COOKIE_SECURE are set in docker-compose.yml, not here
```

> **Tip:** Never commit `.env` to git. It is already in .gitignore.

Step 7. Build and start the stack

From the project folder, in PowerShell:

```
docker compose up -d --build
```

The first build takes several minutes (it downloads Node and PostgreSQL images and compiles the app). Later starts take seconds. The app container runs the database migrations automatically before it starts.

Check that both containers are running:

```
docker compose ps
```

Both **db** and **app** should show **running**, and db should show **healthy**.

Step 8. Confirm the database migrations ran

```
docker compose logs app
```

You should see `Running database migrations...`, then `All migrations have been successfully applied.`, then `Ready in ...`. If migrations fail, the app will not start; the error is printed in this log.

Step 9. Open the CMS

Device	Address to open
The host PC	`http://localhost:3000`
Any phone or laptop on the same WiFi	`http://192.168.29.239:3000` (use your IP from Step 2)
A screen player PC	Set the server to the same address (see Step 11)

On first use, sign in or create the first account. The first account created through the registration form is signed in straight away, and after that only signed-in users can add accounts.

Step 10. Test from another device

On a phone connected to the same WiFi (not the host PC), open `http://<host-IP>:3000`. If the sign-in page loads, the network is working. If the page does not load, go to the Troubleshooting section.

Step 11. Point the screen players at this server

The screen player is the Electron app in the `desktop/` folder. Its server address is read from `desktop/build-config.json`. Change it to your host address and rebuild the player:

```
{
  "serverUrl": "http://192.168.29.239:3000"
}
```

```
cd desktop
npm install
npm run dist
```

The portable exe is written to `desktop/dist/`. Copy it to each screen PC. On the screen PC, open it, enter the **Screen ID** from the CMS Screens page and press Submit. Each screen PC needs to be on the same WiFi as the host.

> **Tip:** Repository CI overwrites `build-config.json` from the CMS_URL GitHub variable when it builds in GitHub Actions. For a local build, edit the file as shown above and build on this PC.

Daily operation

Task	Command (run in the project folder)
Start everything	`docker compose up -d`
Stop everything (data is kept)	`docker compose down`
View app logs	`docker compose logs -f app`
Restart the app only	`docker compose restart app`
Update after a code change	`docker compose build app`, then `docker compose up -d`
Back up the database	`docker compose exec -T db pg_dump -U postgres multimedia > backup.sql`
Open a database shell	`docker compose exec db psql -U postgres multimedia`

> **Warning:** `docker compose down -v` deletes the database volume and all CMS data (sessions, screens, users). Do not run it unless you want to erase everything.

Troubleshooting

Problem	Cause and fix
Other devices cannot open http://IP:3000	(1) The network is Public: set it to Private (Step 3). (2) The firewall rule is missing: re-run Step 4. (3) Client isolation is on: a guest network or some routers stop WiFi devices from reaching each other. Use the main network, or turn off AP/client isolation in the router.
Login succeeds but the page keeps returning to the sign-in page	The session cookie was blocked. Check that `COOKIE_SECURE: "false"` is in docker-compose.yml, then run `docker compose up -d --build`.
Address changed and the players stopped connecting	The host PC got a new IP. Set a DHCP reservation (Step 2), then update build-config.json and rebuild the players.
Port 3000 already in use	Change the left number in docker-compose.yml to another port, e.g. `"3001:3000"`, then use http://IP:3001. Repeat the firewall rule for that port.
app container keeps restarting	Read `docker compose logs app`. **'exec ./entrypoint.sh: no such file or directory'** means the script has Windows (CRLF) line endings. Convert `entrypoint.sh` to LF line endings (in VS Code: bottom-right 'CRLF' → 'LF'), then run `docker compose up -d --build`. The repository's .gitattributes keeps it LF. Other causes are a migration error or a missing .env value.
Upload fails	Check the Cloudinary values in .env, then rebuild and restart (env values are read at start).
Screen shows old content	Changes are sent through Ably if ABLY keys are set. Otherwise the player picks them up on restart.
Docker commands fail with 'cannot connect to the Docker daemon'	Docker Desktop is not running. Start it from the Start menu and wait for the whale icon to settle.

Security notes for a shared WiFi

> **Warning:** Traffic on this setup is plain HTTP. Anyone on the same WiFi can see sign-in details and session cookies on the wire. Use it only on a trusted, private network. For wider access, put the app behind an HTTPS reverse proxy and set COOKIE_SECURE to true (or remove the line).

Change the default passwords before going live. The database password is `postgres` in docker-compose.yml. The preset sign-in accounts in `lib/auth.ts` (ALLOWED_USERS) have passwords written into the source code, and those are in the git history. Rotate them and move them out of the code before you rely on this deployment.

Keep the database private. The compose file does not publish port 5432, so the database stays inside Docker. Do not add a `ports:` entry for it.

Close the firewall rule when the CMS is not needed on the network: `Remove-NetFirewallRule -DisplayName "TR CMS (port 3000)"`.

Quick reference: `docker compose up -d --build` | `docker compose logs -f app` | `http://<host-IP>:3000`